

■ Termux Device Dashboard

Dashboard de Monitorização Pessoal para Android via Termux

Relatório Técnico de Projeto — Portfólio

Autor: Elias Joaquim

Agosto de 2026

Repositório: github.com/eliasjoaquir/termux-device-dashboard

1. Introdução

Este relatório documenta o desenvolvimento do **Termux Device Dashboard**, uma aplicação web local que permite monitorizar em tempo real métricas do próprio dispositivo Android — bateria, ligação Wi-Fi e armazenamento — diretamente através do Termux, sem depender de servidores externos ou serviços na nuvem.

O projeto foi concebido como peça de portfólio, com o objetivo de demonstrar competências em desenvolvimento backend, integração com APIs de sistema Android, criação de interfaces web responsivas e organização de projetos para publicação em repositórios open-source.

2. Objetivos do Projeto

- Criar uma ferramenta funcional de monitorização pessoal executável inteiramente no telemóvel.
- Aplicar conhecimentos de desenvolvimento backend com Python e Flask.
- Integrar dados reais do sistema Android através do pacote termux-api.
- Construir uma interface web simples, responsiva e sem dependências pesadas.
- Publicar o projeto no GitHub como parte do portfólio profissional.

3. Tecnologias Utilizadas

Componente	Tecnologia	Função
Ambiente de execução	Termux	Terminal Linux para Android, onde toda a aplicação corre
Backend	Python 3 + Flask	Servidor web e API que expõe as métricas do sistema
Dados do sistema	termux-api	Acesso a bateria, Wi-Fi e sensores do Android
Frontend	HTML5 / CSS3 / JavaScript	Interface do dashboard, sem frameworks externos
Controlo de versões	Git / GitHub	Versionamento e publicação do código-fonte

4. Arquitetura e Funcionamento

A aplicação segue uma arquitetura cliente-servidor simples, correndo inteiramente em *localhost* no próprio telemóvel:

- **Backend (app.py):** servidor Flask que expõe os endpoints */api/battery*, */api/network*, */api/storage* e */api/all*, cada um invocando comandos do termux-api (ou *df*, no caso do armazenamento) e devolvendo os dados em JSON.
- **Frontend (templates/ e static/):** página HTML que consulta o endpoint */api/all* a cada 5 segundos via JavaScript (polling) e atualiza os indicadores visuais de bateria, rede e armazenamento.
- **Tratamento de falhas:** cada endpoint devolve um erro controlado caso o comando termux-api correspondente não esteja disponível, evitando que o servidor falhe por completo.

5. Funcionalidades Implementadas

- Monitorização da percentagem de bateria, estado de carregamento, temperatura e saúde.
- Visualização da rede Wi-Fi ligada, incluindo força do sinal e velocidade de ligação.
- Indicador de uso de armazenamento interno, com alerta visual quando próximo do limite.
- Atualização automática dos dados sem necessidade de recarregar a página.
- Interface adaptada a ecrãs de telemóvel, com modo escuro.

6. Publicação no GitHub

O código-fonte foi organizado em repositório próprio e publicado no GitHub, com estrutura de pastas separando backend, templates e ficheiros estáticos, incluindo documentação (README) com instruções completas de instalação no Termux.



Figura 1 — Repositório publicado no GitHub via telemóvel.

7. Desafios e Aprendizagens

- Configuração do ambiente Termux e permissões necessárias para o termux-api funcionar corretamente com a app Termux:API.
- Tratamento de casos em que determinados sensores ou dados não estão disponíveis, garantindo que o dashboard continua a funcionar de forma parcial.

- Organização do fluxo de publicação no GitHub diretamente a partir do telemóvel, sem acesso a computador.

8. Melhorias Futuras

- Adicionar histórico de métricas com base de dados SQLite e gráficos de evolução.
- Implementar autenticação simples para proteger o acesso na rede local.
- Incluir notificações automáticas de bateria fraca via termux-notification.
- Expandir para sensores adicionais (luz, acelerómetro, localização).

9. Conclusão

O projeto cumpriu o objetivo de criar uma ferramenta funcional de monitorização pessoal, inteiramente desenvolvida e executada num ambiente móvel através do Termux. Para além do resultado técnico, o processo permitiu consolidar competências em Python, desenvolvimento web, integração de APIs de sistema e boas práticas de publicação de código em repositórios públicos — todas relevantes para um percurso profissional em desenvolvimento de software.